

Time Complexity Analysis - Radius Search Methods

Contents

1	Method Descriptions and Complexity Analysis	2
1.1	radiusSearch(centerLat, centerLon, radiusKm)	2
1.2	radiusSearchHelper(node, centerLat, centerLon, radiusKm, depth, result)	2
1.3	getRadiusSearchSummary(centerLat, centerLon, radiusKm)	2
1.4	getRadiusSearchAVL(centerLat, centerLon, radiusKm)	3
2	Time Complexity Summary	4

1 Method Descriptions and Complexity Analysis

1.1 radiusSearch(centerLat, centerLon, radiusKm)

Description:

- Validates the input coordinates and radius
- Initializes a result list
- Calls the recursive `radiusSearchHelper` to traverse the KD-tree
- Returns all nodes whose coordinates fall within the specified radius using Haversine distance

Time Complexity

Average: $O(k + \log n)$

Worst Case: $O(n)$

1.2 radiusSearchHelper(node, centerLat, centerLon, radiusKm, depth, result)

Description: This method performs the actual KD-tree radius search.

For each node:

- Terminates if the node is null
- Extracts its latitude and longitude
- Computes the Haversine distance to the center point
- Adds the node to results if inside radius
- Determines splitting axis using `depth % 2`
- Chooses which subtree to explore first
- Computes distance to splitting plane
- Explores opposite subtree if plane intersects search radius

Time Complexity

Average: $O(k + \log n)$

Worst Case: $O(n)$

1.3 getRadiusSearchSummary(centerLat, centerLon, radiusKm)

Description:

- Calls `radiusSearch` to get nodes within radius
- Groups stations found by country
- Stores distance to each station from center

- Sorts stations within each country
- Builds formatted summary with:
 - Search center information
 - List of countries and stations
 - Distance, location, time zone, and attributes

Time Complexity

$O(s \log s)$

1.4 `getRadiusSearchAVL(centerLat, centerLon, radiusKm)`

Description:

- Executes the radius search
- For each resulting node:
 - Creates a copy including the distance
 - Iterates through each station in that node:
 - * Searches for its coordinates in the AVL tree
 - * If already in the AVL, adds the station to the existing node
 - * Otherwise inserts a new node into the AVL

Produces an **AVL tree that contains only the stations inside the radius.**

Time Complexity

$O(s \log s)$

2 Time Complexity Summary

Method	Average Case	Worst Case
radiusSearch	$O(k + \log n)$	$O(n)$
radiusSearchHelper	$O(k + \log n)$	$O(n)$
getRadiusSearchSummary	$O(s \log s)$	
getRadiusSearchAVL	$O(s \log s)$	

Where:

n = total nodes in the KD-tree

k = nodes within radius

s = total stations inside those nodes